GOLDEN RASTREAMENTO

Instalação completa do sistema

Site, captação de leads, painel (CRM, CMS e ERP), loja virtual e portal do cliente — do primeiro `git clone` ao checklist de saída.

Versão **2.21.4** Data **24/10/2026** Requisito **PHP 8.3 + MySQL**

Responsável **Marcelo · KDKHOST** Contato **contato@kdkhost.com.br**

1. O que a instalação faz

1. Confere PHP, extensões e Composer antes de mexer em qualquer coisa.
2. Cria o arquivo `.env` de produção (chave da aplicação, endereço, banco, fuso horário, filas e cache) com permissão `600`.
1. Instala as dependências do projeto sem as bibliotecas de desenvolvimento.
2. Prepara o banco: se ele já tiver tabelas de outro sistema, tira uma cópia de segurança e recria as tabelas do Golden a partir das migrations.
1. Grava os dados de demonstração (catálogo, páginas do site, leads de exemplo) — ou só o catálogo base, com `--sem-demo`.
1. Define o login do painel.
2. Cria o link `public/storage` para as imagens, vídeos e documentos enviados.
3. Escreve um `.htaccess` na raiz que bloqueia código-fonte, `.env` e logs, faz todo pedido entrar pela pasta `public` e, com `--php-web=`, escolhe o PHP que serve a web.
1. Liga os caches de configuração, rotas e views.

2. Abre o próprio `--url` no final e avisa se o PHP da web for outro.

Nada de credencial fica no repositório: o instalador perguntou na tela ou leu de uma variável de ambiente e gravou apenas no `.env`, que está no `.gitignore`.

---

2. Requisitos

Item	Mínimo	Como conferir
PHP	8.3	`php -v`
Extensões obrigatórias	pdo, mbstring, openssl, tokenizer, xml, fileinfo, curl, json	`php -m`
Extensões recomendadas	gd, intl, zip, exif, pdo_mysql	`php -m`
Composer	2.x	`composer --version`
MySQL	5.7 / MariaDB 10.3+	—
Apache com `mod_rewrite`	—	necessário quando o docroot é a raiz do projeto
Limite de envio	`upload_max_filesize` e `post_max_size` ≥ 40 MB	`php -i | grep upload_max`

O `gd` é o que permite ler a largura e a altura da imagem no campo de envio; sem ele o upload funciona, mas a dica de dimensão fica vazia. `intl` e `zip` são exigidos por pacotes do Composer; `exif` ajuda a validar foto de celular.

PHP 8.3 é inegociável

O `composer.json` pede `php: ^8.3` (Laravel 13). Se o servidor responder com

`Composer detected issues in your platform: Your Composer dependencies require a PHP version ">= 8.3.0". You are running 8.1.34.`

o site inteiro fica em erro 500, mesmo com banco e arquivos corretos. O ponto é que existem **dois PHPs no servidor**: o do comando `php` (linha de comando) e o que atende o navegador. Mudar um não muda o outro, e é o segundo que quebra o site.

- **cPanel:** *Software → MultiPHP Manager* → marque o domínio → *PHP Version*

→ escolha 8.3 (ou 8.4) → *Apply*. Depois confira em *MultiPHP INI Editor* os limites de `upload_max_filesize` e `post_max_size`.

- **cPanel com CloudLinux + LiteSpeed (o caso mais chato):** o *Select PHP

Version* da conta é um seletor próprio e pode parar no 8.2 mesmo com o 8.3 instalado no servidor. Nesse caso o `.htaccess` resolve, sim, desde que use o handler certo. Acrescente esta linha ao `.htaccess` da raiz do projeto (ou do `public/`, se o docroot for ele):

``` `apache AddType application/x-httpd-ea-php83 .php` ```

Foi exatamente assim que o 8.1 virou 8.3 num servidor cPanel + CloudLinux + LiteSpeed: o seletor da conta não oferecia 8.3 (`cloudlinux-selector get --interpreter php --get-supported-versions` parava em 8.2), mas o `ea-php83` estava instalado e o handler assumiu os arquivos `.php`. O instalador faz isso sozinho com `--php-web=ea-php83`.

Confira depois com um arquivo temporário dentro de `public/`:

``` `php <?php echo PHP_VERSION . ' | ' . PHP_SAPI; // salve como public/_chk.php` ```

Abra `https://seudominio/_chk.php`, confirme `8.3.x | litespeed` e **apague o arquivo** em seguida. Se aparecer `apachectl/lsapi` diferente ou a versão antiga, o handler escolhido não existe no servidor — liste os disponíveis com `ls -d /opt/cpanel/ea-php*`.

- **Extensões que costumam faltar no PHP da web:** `fileinfo` (upload de

imagem, vídeo e documento) e `intl`. No seletor do CloudLinux elas se acendem pela linha de comando, com o nome de estado por extensão — e a lista **substitui** a anterior, então informe tudo de uma vez:

``` `sh cloudlinux-selector set --json --interpreter php --version 8.3 --user SEU_USUARIO \ --extensions '{"fileinfo":"enabled","intl":"enabled","gd":"enabled","zip":"enabled","pdo_mysql":"enabled","mbstring":"enabled","bcmath":"enabled","dom":"enabled"}'` ```

- **Plesk:** *PHP Versions / Settings* → PHP 8.3.
- **VPS (Ubuntu):** `apt install php8.3-fpm php8.3-mysql php8.3-gd php8.3-intl php8.3-zip`

e aponte o `fastcgi_pass` do nginx para o 8.3.

O instalador detecta a versão do binário que você passou (`--php=`) e **para antes** de instalar qualquer coisa se ela for menor que 8.3. No final ele ainda abre o `--url` informado: se a home responder o erro de plataforma, ele avisa na tela que o PHP **da web** é outro e mostra a linha `--php-web=` que resolve.

---

3. Instalação com o instalador (recomendado)

Coloque o código no servidor (git ou upload do `.zip`), entre na pasta do projeto e rode:
```

```
GOLDEN_DB_PASSWORD='senha-do-banco' GOLDEN_ADMIN_PASSWORD='senha-do-painel' \
  bash deploy/instalador-golden.sh \
    --url=https://seudominio.com.br \
    --nome=nome_do_banco \
    --usuario=usuario_do_banco
```

Passar as senhas por variável de ambiente evita que elas fiquem no histórico do shell; sem as variáveis, o instalador pede na tela sem mostrar o que foi digitado.

Opções do instalador

Opção	Para que serve
`--url=`	endereço público do site, com `https://` (obrigatório)
`--banco=mysql|sqlite`	motor do banco; padrão `mysql`
`--host=`, `--porta=`	servidor MySQL; padrão `127.0.0.1:3306`
`--nome=`	nome do banco (no SQLite, o arquivo)
`--usuario=`, `--senha=`	credenciais do banco
`--criar-banco`	cria o banco se não existir (precisa de privilégio `CREATE`)
`--admin-email=`, `--admin-nome=`, `--admin-senha=`	login do painel
`--sem-demo`	não grava os dados de demonstração
`--limpar-banco`	apaga todas as tabelas antes de recriar (sem a flag, ele pergunta)
`--sem-backup`	não tira cópia do banco antes de limpar
`--dev`	`APP_ENV=local` e `APP_DEBUG=true` (nunca em produção)
`--sem-htaccess`	não escreve o `.htaccess` de proteção da raiz
`--sem-cache`	não gera cache de configuração, rotas e views
`--php=`, `--composer=`	força o binário quando há mais de uma versão no servidor
`--php-web=ea-php83`	grava no `.htaccess` o handler do PHP que serve a web (CloudLinux/LiteSpeed)
`--help`	tela de ajuda completa

Exemplos prontos

cPanel, reaproveitando um banco que já existe no plano

```
GOLDEN_DB_PASSWORD='...' GOLDEN_ADMIN_PASSWORD='...' \
  bash deploy/instalador-golden.sh \
    --url=https://seudominio.com.br \
    --php=/usr/local/bin/ea-php83 --php-web=ea-php83 \
    --nome=usuario_do_banco --usuario=usuario_do_banco \
    --limpar-banco
```

VPS nova, criando o banco

```
bash deploy/instalador-golden.sh --url=https://painel.exemplo.com.br \
  --nome=golden --usuario=golden --criar-banco
```

Teste local, sem MySQL

```
bash deploy/instalador-golden.sh --url=http://localhost:8000 \
  --banco=sqlite --dev --sem-htaccess
```

O que aparece na tela

Cada etapa imprime uma linha com ✓. Se algo falhar, o instalador para, mostra o comando que falhou e as últimas 25 linhas do log (`storage/logs/instalador-*.log`), que guarda a saída completa. No fim ele imprime o resumo com endereço, painel, banco, ambiente e caminho do log.

Limpar um banco que já tem outro sistema

O instalador conta as tabelas antes de encostar. Havendo tabelas, ele:

1. faz `mysqldump` comprimido para a pasta **acima** do projeto — fora do que o

navegador serve — com permissão `600`;

1. pede confirmação explícita (digitar `SIM`), ou exige a flag `--limpar-banco`

quando roda sem terminal;

1. só então roda `migrate:fresh`.

Para restaurar a cópia depois:

```
gunzip < ../backup-banco-nome-20261023-1130.sql.gz | mysql -u USUARIO -p NOME_DO_BANCO
```

---

4. Instalação manual, passo a passo

Serve para qualquer servidor e é o roteiro usado quando o instalador não pode rodar (por exemplo, hospedagem sem SSH).

4.1 Subir o código

- **Com git:** `git clone https://github.com/kdkhost/GOLDENRASTREAMENTO.git public_html`
- **Sem git:** baixe o `.zip` do repositório e extraia na pasta do domínio.

Confirme que `artisan`, `app/`, `public/` e `composer.json` estão na raiz da pasta extraída.

4.2 Criar o banco (cPanel)

1. *MySQL® Databases* → criar banco; a hospedagem costuma prefixar com o nome

da conta, ex.: `seuusuario_cms`. Anote o nome completo.

1. Criar usuário e **senha forte**; anote os dois.
2. Adicionar o usuário ao banco com **ALL PRIVILEGES**.
3. *phpMyAdmin* → conferir que o banco aparece e está vazio.

Se aparecer `Access denied for user`, faltou ligar o usuário ao banco.

4.3 Criar o `.env`

```
cp .env.example .env
php artisan key:generate
chmod 600 .env
```

Ajuste, no mínimo:

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://seudominio.com.br

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=seuusuario_cms
DB_USERNAME=seuusuario_cms
DB_PASSWORD=senha-do-banco
```

Veja a referência completa de todas as chaves na seção 7.

4.4 Dependências

```
composer install --no-dev --prefer-dist --optimize-autoloader
```

Sem Composer no PATH, baixe o arquivo único e use pelo PHP:

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php composer-setup.php --install-dir=$HOME/bin --filename=composer.phar
php $HOME/bin/composer.phar install --no-dev --prefer-dist --optimize-autoloader
```

4.5 Banco de dados

```
php artisan migrate --force           # só as tabelas
php artisan migrate --force --seed    # com catálogo, páginas e demonstração
```

Para começar do zero em um banco que já tinha outra coisa:

```
php artisan migrate:fresh --seed --force
```

4.6 Imagens, vídeos e documentos

```
php artisan storage:link
```

Cria `public/storage` apontando para `storage/app/public`. Se o link não puder ser criado (algumas hospedagens bloqueiam o `symlink`), faça manualmente:

```
ln -s ../storage public/storage
```

Permissões: pastas `755`, arquivos `644`, e `storage/` + `bootstrap/cache/` precisam ser graváveis pelo usuário do webserver.

Sem esse link, **as fotos enviadas do painel não aparecem** — nem no site (`storage/site/…`, administrado em *Conteúdo → Imagens do site*), nem na loja e nos pedidos. O que não foi enviado pelo painel continua servindo de `public/img/`, então o site abre e só as fotos trocadas ficam quebradas: é o sintoma de `storage:link` faltando.

4.7 Entrega pelo docroot

Quando o domínio aponta para a **raiz do projeto** (caso típico do cPanel, `public_html`), use o `.htaccess` gerado pelo instalador — ele:

- desliga o índice de diretório (`Options -Indexes`), o que impede a listagem do

código-fonte;

- bloqueia `.env`, `.env.*`, `.git*`, `artisan`, `composer.*`, `package.json`,

`vite.config.js`, `phpunit.xml`, `error_log` e qualquer `.log`;

- bloqueia as pastas `app`, `bootstrap`, `config`, `database`, `resources`,

`routes` e `tests`;

- reescreve todo pedido para dentro de `public/`.

`storage` **não** entra na lista de bloqueio: o que o público alcança é o link `public/storage`, por onde saem as imagens, vídeos e documentos enviados pelo painel. Bloquear o caminho `/storage` na raiz devolve 403 em todo arquivo enviado, e o site fica com as imagens quebradas. A privateidade do `storage/` de verdade vem da reescrita (nada alcança `storage/logs`, `storage/framework` ou `storage/app/private`) e do bloqueio de `.log`.

O teste de barreira, já com o site no ar:

```
curl -I https://seudominio.com.br/.env          # 403
curl -I https://seudominio.com.br/app/          # 403
curl -I https://seudominio.com.br/storage/logs/laravel.log   # 403
curl -I https://seudominio.com.br/              # 200
curl -I https://seudominio.com.br/loja          # 200
```

E o teste do que o painel envia, que é o que costuma sobrar depois de mexer no `.htaccess`: envie uma imagem pelo painel e abra a URL devolvida no JSON de `/admin/uploads`. Tem de ser `200` com o `Content-Type` do arquivo — se der 403, a regra de bloqueio está alcançando `public/storage`.

Quando o docroot é a pasta `public/` (VPS com nginx, ou subdomínio apontando para `public`), o `.htaccess` da raiz nem é lido — e a proteção já existe por construção.

4.8 Caches de produção

```
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Depois de qualquer mudança em `.env`, `config/`, `routes/` ou `resources/views/` rode os três de novo (ou `php artisan optimize:clear` e recarregue).

4.9 Primeiro acesso

- Site: `https://seudominio.com.br`
- Painel: `https://seudominio.com.br/admin` — login definido na instalação.
- Portal do cliente: `https://seudominio.com.br/portal/entrar` — ninguém precisa

criar conta no painel; o auto-cadastro reaproveita a ficha do CRM (seção 10).

- Troque a senha padrão assim que entrar, e remova o usuário de demonstração

(`vendedor@golden.com.br`) se não for usar.

---

5. Instalação no cPanel, em detalhe

1. *MultiPHP Manager* → PHP **8.3** para o domínio. Se o plano usar CloudLinux,

confira também o *Select PHP Version* da conta; quando ele não chega no 8.3, pule essa etapa e deixe o `.htaccess` escolher o handler (`--php-web=ea-php83`, seção 2).

1. *MySQL® Databases* → banco + usuário + privilégios (seção 4.2).
2. *Git™ Version Control* (se houver): *Clone Repository* com a URL pública do

repositório e o branch `main`, caminho `public_html`. Não use credencial na URL: o repositório é público. Sem o Git™, envie o `.zip` pelo *Gerenciador de Arquivos* e extraia.

1. *Terminal* (ou SSH com chave) → na pasta do projeto:

`git pull`, depois o instalador:

``` `bash GOLDEN_DB_PASSWORD='...' GOLDEN_ADMIN_PASSWORD='...' \ bash deploy/instalador-golden.sh \ --url=https://seudominio.com.br \ --php="$(command -v ea-php83)" --php-web=ea-php83 \ --nome=usuario_mysql --usuario=usuario_mysql --limpar-banco` ```

1. *Gerenciador de Arquivos* → confira as permissões: `.env` em `600`,

pastas `755`.

1. *MultiPHP INI Editor* → `upload_max_filesize=40M`, `post_max_size=45M`,

`max_execution_time=120` (o campo de vídeo aceita até 40 MB).

1. Abra o site e faça o teste de barreira da seção 4.7.
2. *SSL/TLS → Let's Encrypt* (ou o auto-SSL da hospedagem) ativo, e

Forçar HTTPS ligado.

Para outros agentes ou para usar de outra máquina, a mesma conta pode ser acessada por chave SSH: gere `ssh-keygen -t ed25519 -f ~/.ssh/golden_deploy -N ""`, cole a `.pub` em *Segurança → Chaves SSH / Access Hash*, e crie um apelido no `~/.ssh/config`. **Nunca** coloque senha, chave ou `.env` dentro do repositório.

```

6. Atualização depois da primeira instalação

```
bash deploy/atualiza-golden.sh                  # ramo main
bash deploy/atualiza-golden.sh --ramo=main --php=/usr/local/bin/ea-php83
```

Ele faz `git pull --ff-only`, `composer install --no-dev`, `migrate --force`, `storage:link` e reativa todos os caches. Se uma migration nova existir, ela é aplicada sem tocar nos dados.

Quando mexer em `.env` à mão:

```
php artisan config:clear && php artisan config:cache
```

---

7. Referência do `.env`

Chave	Produção	Observações
`APP_NAME`	`Golden Rastreamento`	nome em e-mails e no painel
`APP_ENV`	`production`	nunca `local` em servidor
`APP_DEBUG`	`false`	`true` vaza pilha de erro para o visitante
`APP_URL`	`https://seudominio.com.br`	usado em links de e-mail e assets
`APP_KEY`	gerado por `key:generate`	se trocar, sessões e dados criptografados somem
`APP_TIMEZONE`	`America/Sao_Paulo`	datas de pedidos e leads
`APP_LOCALE` / `APP_FALLBACK_LOCALE`	`pt_BR`	textos e moeda
`DB_CONNECTION`	`mysql`	`sqlite` apenas em teste
`DB_HOST`, `DB_PORT`	`127.0.0.1`, `3306`	no cPanel costuma ser `localhost`
`DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`	do painel	nunca no git
`SESSION_DRIVER`	`file`	sessões não geram consulta ao banco
`CACHE_STORE`	`file` (ou `redis`)	
`QUEUE_CONNECTION`	`database`	exige `php artisan queue:work`
`FILESYSTEM_DISK`	`public`	onde os envios vão parar
`LOG_CHANNEL` / `LOG_LEVEL`	`stack` / `warning`	log em `storage/logs`
`MAIL_MAILER`	`smtp`	use `log` enquanto não tiver credencial
`MAIL_LOG_CHANNEL`	`email`	para onde o `log` escreve o e-mail (`storage/logs/email.log`)
`MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_ENCRYPTION`	do seu servidor de correio	o **código de e-mail está pronto no projeto**; falta só a credencial do servidor

Chave	Produção	Observações
`MAIL_FROM_ADDRESS`, `MAIL_FROM_NAME`	`disparos@foco.server-golden.com.br` / `Golden Rastreamento`	remetente dos orçamentos e avisos: caixa de máquina, não o e-mail de uma pessoa
`GOLDEN_WHATSAPP`	`5519997101483`	botão de WhatsApp do site
`GOLDEN_EMAIL`	endereço comercial	aparece no site e nos avisos
`GOLDEN_TRACKING_ENABLED`	`0` ou `1`	liga a consulta de rastreio
`GOOGLE_BUSINESS_ACCOUNT`, `GOOGLE_BUSINESS_LOCATION`, `GOOGLE_BUSINESS_TOKEN`	do seu perfil	depoimentos do Google com avatar; sem eles o importador só avisa (seção 11)
`GOOGLE_BUSINESS_LIMITE`, `GOOGLE_BUSINESS_NOTA_MINIMA`	`50`, `4`	até quantas avaliações ler e a nota mínima aceita

Sobre os e-mails: os modelos, o envio de orçamento e as notificações de pedido já existem. Trocar `MAIL_MAILER=log` para `smtp` e preencher as cinco chaves é tudo que falta para eles saírem de verdade. Em `log`, a mensagem inteira é escrita em `storage/logs/email.log` — útil para conferir o conteúdo antes de liberar o SMTP. Esse canal é separado do log da aplicação de propósito: o driver `log` grava o aviso em nível `debug`, e uma produção com `LOG_LEVEL=warning` filtrava essas linhas — o `queue:work` dizia `DONE` e não sobrava rastro nenhum do envio.

A caixa de disparo se cria pela SSH, sem abrir o painel

Não use o e-mail de ninguém do dia a dia como remetente do sistema: crie uma caixa de máquina, só para conexão e disparo. Com SSH na hospedagem cPanel:

```
uapi Email add_pop email=disparos@seudominio.com.br password='SENHA_FORTE' quota=0
uapi Email list_pops          # confere que a caixa existe
```

Três coisas que só aparecem na hora:

- **o domínio da caixa tem de ser um que a conta hospeda.** A API responde

`you do not have access to the domain 'outrodominio.com.br'` para domínio que está em outra conta do mesmo servidor. `uapi Email list_mail_domains` lista os liberados.

- **SPF e DKIM precisam cobrir esse domínio.** Confira `TXT seudominio.com.br` e

`TXT default._domainkey.seudominio.com.br`; sem isso a mensagem sai e cai em spam.

- **senha com `#` no `.env` só funciona entre aspas.** Sem as aspas, o `#` começa um

comentário e o `MAIL_PASSWORD` chega vazio ao Laravel — o erro aparece como falha de login no SMTP, não como erro de arquivo.

Depois, `MAIL_MAILER=smtp` no `.env` e uma prova curta antes de mexer no site:

```
Mail::raw('prova', fn ($m) => $m->to('voce@seudominio.com.br')->subject('prova'));
```

Se o envio for aceito, a mensagem aparece na caixa de destino; o `queue:work` do cron (seção 6) cuida do resto sem intervenção.

---

8. Loja virtual

A loja (catálogo, carrinho, checkout e pedidos no painel) vem ligada. Para desligar a vitrine sem desligar o resto, use o painel em *ERP → Configurações da loja* (chaves `loja_ativa` e `loja_livre`); o site some do menu e o `/loja` responde 404, mas os pedidos já registrados continuam no painel.

Os campos de envio do painel (imagem do produto, vídeo do produto, documentos do pedido e o PDF do contrato) aceitam arrastar-e-soltar, mostram preview, progresso real, tempo decorrido e a regra do campo (formatos, tamanho máximo e dimensão ideal). As regras moram em `config/envios.php`:

Campo	Formatos	Limite	Pasta no disco
`produto_imagem`	JPG, JPEG, PNG, WEBP	4 MB	`produtos/`
`produto_video`	MP4, WEBM	40 MB	`produtos/video/`
`pedido_documento`	PDF, JPG, JPEG, PNG	10 MB cada, até 12 por pedido	`pedidos/`
`contrato_documento`	PDF	8 MB	`contratos/`

O servidor confere a extensão **do conteúdo** do arquivo, não o nome que o navegador mandou, e recusa o envio se o tipo não estiver entre os formatos do campo. Por isso os limites de `upload_max_filesize` / `post_max_size` da seção 2 importam: arquivo maior que isso chega cortado e o campo avisa o limite do servidor.

---

9. Tráfego e atribuição

O sistema identifica a origem de **cada visita, clique e cadastro** sem depender de Google Analytics: as tabelas `visits` e `visit_events` gravam a jornada, e o painel abre em *Comercial → Atribuição de origens* (`/admin/atribuicao`) com as tabelas por canal, rede, campanha, página e dispositivo; a ficha do lead (`/admin/leads/12`) mostra primeiro toque, último toque e a linha do tempo da visita.

Nada precisa ser configurado para funcionar: os sinais vêm da URL (`utm_*`, `gclid`, `fbclid`, `ttclid`, `msclkid`, `li_fat_id`, `yclid`), do referer e do user-agent. As listas que mandam na classificação estão

em `config/golden.php` (`canais`, `meios_pagos`, `meios_sociais`, `meios_ia`, `buscadores`, `ia_usuarios`, `indexadores`) — acrescentar uma rede nova é editar o config, não o código.

O cookie de visitante (`golden_visitor`, 90 dias) é de primeira parte e não guarda dado pessoal; o IP entra como hash. Robôs de índice (`GPTBot`, `ClaudeBot`, `Meta-ExternalAgent`, o crawler do Google…) entram no registro mas ficam **fora de todo número** — já o navegador embutido do WhatsApp/Instagram, onde o clique humano aconteceu, conta como gente.

Retenção e agendamento

O registro cresce com o tráfego, por isso há poda:

```
php artisan rastreio:podar            # usa golden.rastreio.dias_retencao (365 dias)
php artisan rastreio:podar --dias=90  # período menor, quando preciso
```

A poda roda toda terça às 03:30 pelo agendador do Laravel (`bootstrap/app.php`), que precisa de **uma** linha no cron da hospedagem (cPanel → *Tarefas de Cron*):

```
* * * * * cd ~/public_html && /opt/cpanel/ea-php83/root/usr/bin/php artisan schedule:run
>> /dev/null 2>&1
```

O caminho do PHP é o **8.3 do site**, não o `php` do PATH da conta (que costuma ser mais velho). Para descobrir o certo no seu servidor: `ea-php83 -r 'echo PHP_VERSION;'` ou `ls /opt/cpanel/ea-php8*/root/usr/bin/php`. Sem o cron, o comando continua podendo ser rodado à mão — só não se auto-agenda.

A mesma linha de cron também drena a fila de e-mails. Todo aviso do portal (chamado aberto, resposta da equipe, recibo, link de recuperação de senha) entra em fila, e o agendador dispara `queue:work --stop-when-empty --max-time=50` a cada minuto. Numa hospedagem sem esse cron as mensagens ficam paradas na tabela `jobs` e o cliente espera um e-mail que nunca sai. Para conferir à mão:

```
php artisan queue:work --stop-when-empty   # consome o que está preso e sai
```

Não use `queue:listen` nem deixe um worker residente: a hospedagem compartilhada não tem supervisor, e o `--stop-when-empty` existe justamente para o trabalho caber no disparo de um minuto do `schedule:run`.

Se um aviso estourar ao ser montado (endereço inválido, dado faltando), ele não se perde: sai de `jobs` e entra em `failed_jobs`, com o erro por extenso. Depois de corrigir a causa, o retry recoloca a mensagem na fila sem precisar abrir o chamado de novo:

```
php artisan queue:retry all    # devolve tudo que está em failed_jobs para a fila
```

---

10. Portal do cliente

O portal (`/portal`) é a casa do cliente no mesmo domínio do site: ele vê os próprios pedidos, abre e acompanha chamados, lê os contratos publicados e corrige o cadastro — sem depender da equipe e sem nunca receber acesso ao painel.

Tela	Endereço
Entrar	`/portal/entrar`
Criar conta (auto-cadastro)	`/portal/cadastro`
Esqueci a senha / redefinir	`/portal/esqueci-senha`, `/portal/redefinir-senha`
Início (resumo)	`/portal`
Pedidos	`/portal/pedidos`, `/portal/pedidos/{id}`
Chamados	`/portal/chamados`, `/portal/chamados/abrir`
Contratos	`/portal/contratos`
Meus dados	`/portal/dados`

A entrada aparece no menu do site, na gaveta do celular e na coluna *Suporte* do rodapé, sempre com o mesmo link: `/portal/entrar` para quem ainda não entrou e `/portal` para quem já tem sessão.

A conta nasce sozinha

Ninguém precisa cadastrar cliente no painel para ele entrar. O auto-cadastro procura a ficha do CRM primeiro pelo **documento** (com ou sem máscara) e depois pelo **e-mail**; achou, entrega a senha a ela, e a ficha ganha as quatro colunas de acesso (`email_portal`, `password`, `remember_token`, `ultimo_acesso_em`). Entre duas candidatas, a que tem mais pedidos ganha. Não achou nada, cria a ficha nova, já ativa — e os pedidos feitos como convidado na loja são puxados pelo e-mail, então a primeira tela já vem cheia.

Isso é o que mais gera chamado de suporte: o login do portal é o `email_portal`, e não o e-mail da ficha do CRM. São colunas diferentes de propósito, porque o CRM aceita e-mail repetido entre fichas e a conta do portal não. A ficha do painel (*Clientes → qualquer ficha*) mostra o selo *conta ativa* com o e-mail de acesso e o último acesso, ou *sem conta* — é esse e-mail que funciona no `/portal/entrar`.

Sessão separada, por construção

O portal autentica na guarda `cliente` sobre a tabela `customers`; o painel, na guarda padrão sobre `users`. As duas sessões têm chaves próprias (`login_cliente` / `login`), então abrir o portal não abre rota do painel nem o contrário — dá para ter as duas no mesmo navegador, logadas em pessoas diferentes. Recurso de outro cliente (pedido, chamado ou contrato por id) devolve **404, não 403**: a barra de endereços não descobre o que existe.

O que a equipe faz com isso

- **Admin → Atendimento → Chamados**: KPIs de abertos, sem resposta, aguardando

o cliente e resolvidos; filtro por status e prioridade; responsável; resposta e mudança de status na tela do chamado. A conversa é uma fila só (`tickets` + `ticket_messages`) — a primeira mensagem é o relato que abriu o chamado —, então não há "última resposta" para sincronizar entre as duas telas.

- **Admin → Atendimento → Contratos**: cadastra o documento, anexa o PDF assinado

no dropzone (campo `contrato_documento`: PDF, 8 MB, pasta `contratos/`) e controla a visibilidade pelo status. **Rascunho não aparece para o cliente** — nem na lista, nem no resumo, nem no download. Número comercial `CT26-0001`, vigência com aviso de vencimento em 30 dias.

O PDF do contrato **não** sai por URL pública de `/storage`: o download passa por `/portal/contratos/{id}/arquivo`, que confere se o contrato é da ficha logada e se não é rascunho antes de entregar. Um caminho de arquivo escrito à mão no formulário é recusado — a gravação revalida contra a pasta do campo de envio.

E-mails e recuperação de acesso

Quatro avisos saem do portal: `ChamadoRecebido` (para o cliente), `ChamadoEquipe` (para o painel), `ChamadoResposta` (bidirecional) e `RedefinirSenhaPortal`. Como no resto do sistema, **sem SMTP configurado o aviso se perde e a ação do cliente nunca é barrada** — o envio é tentado em `try/catch`; com `MAIL_MAILER=log` a mensagem fica em `storage/logs/email.log`.

A recuperação de senha não usa tabela de tokens nem broker: o link é assinado (`URL::temporarySignedRoute`) e expira em 60 minutos. Consequências práticas:

- a assinatura depende da `APP_KEY` — trocar a chave invalida links já enviados,

o que é o comportamento desejado, não um bug;

- o e-mail com ou sem conta recebe **a mesma frase**, para a tela não revelar quem

é cliente da Golden;

- o throttle é próprio por rota (cadastro `8,1`, login `10,1`, recuperação `6,1`,

abrir chamado `12,1`) e o cadastro tem honeypot (`website`) mais consentimento LGPD obrigatório.

Migrações novas

Duas migrations entram na atualização: `2026_10_24_000001_cria_conta_do_portal` (as quatro colunas em `customers`) e `2026_10_24_000002_create_tickets_and_contracts_tables` (`tickets`, `ticket_messages`, `contracts`). São aditivas e cada uma tem `down()`; `php artisan migrate --force` as aplica sem tocar nos dados existentes. Toda coluna de texto tem comprimento explícito e `email_portal` fica em 190 — o MySQL da hospedagem não aceita índice único maior, e o SQLite dos testes não avisa.

---

11. Depoimentos do site: painel, Google e portal

O carrossel da home é uma lista só — a tabela `testimonials` — e cada linha diz de onde veio no campo `fonte`:

`fonte`	Quem escreve	Vai ao ar?	Foto do autor
`site`	a equipe, em *Conteúdo → Depoimentos*	quem digita decide	upload opcional no formulário
`google`	`php artisan depoimentos:google` ou o botão **Atualizar do Google**	entra aprovado, com a nota da avaliação	baixada do perfil para `storage/app/public/site/depoimentos`
`cliente`	o próprio cliente, no portal (*Deixar um depoimento*)	**sempre nasce oculto**, aguardando a equipe	a inicial do nome

No painel as três aparecem misturadas na mesma tela, com o selo da origem, o filtro **Origem** e um aviso no topo contando quantos ainda aguardam aprovação. No site, `Testimonial::aprovadosParaSite()` só entrega quem tem `aprovado = true`, ordenado por `ordem` e depois pela data — é por isso que a mesclagem não tem chave nem configuração: aprovar é o único interruptor.

Ler as avaliações do Google

A fonte é a *Business Profile API v4* (`accounts/<ID>/locations/<ID>/reviews`) — a única que devolve a foto de quem avaliou; o Places API comum só entrega avaliações para conta Enterprise. Antes de funcionar, três coisas precisam existir:

1. o perfil verificado no Google Business Profile há mais de 60 dias;
2. a *Business Profile API* liberada para o seu projeto no Google Cloud — a

aprovação do Google leva dias úteis;

1. um Bearer token de OAuth de uma conta que administra esse perfil, com o escopo

`https://www.googleapis.com/auth/business.manage`. Com o `gcloud` instalado:

```
gcloud auth application-default login --
scopes=https://www.googleapis.com/auth/business.manage
gcloud auth application-default print-access-token
```

As três peças vão no `.env` da instalação — nunca no banco e nunca no repositório, que é público:

Chave	Valor
`GOOGLE_BUSINESS_ACCOUNT`	o ID da conta, `accounts/123456789` (ou só `123456789`)
`GOOGLE_BUSINESS_LOCATION`	o ID da unidade, `locations/987654321` (ou só `987654321`)
`GOOGLE_BUSINESS_TOKEN`	o Bearer de OAuth — expira e precisa ser renovado
`GOOGLE_BUSINESS_LIMITE`	avaliações lidas por rodada (padrão `50`)
`GOOGLE_BUSINESS_NOTA_MINIMA`	nota mínima que vira depoimento (padrão `4`)

Com elas preenchidas:

```
php artisan depoimentos:google --apenas-leia   # mostra o que a API devolve, sem gravar
php artisan depoimentos:google                 # importa e atualiza
php artisan depoimentos:google --limite=25
```

A importação é **idempotente**: cada avaliação fica guardada pelo `reviewId` na coluna `externo_id`, então rodar de novo atualiza texto, nota e foto em vez de criar um segundo depoimento da mesma avaliação. Quem não tem texto não entra (sai na contagem `sem_texto`), nem quem está abaixo de `GOOGLE_BUSINESS_NOTA_MINIMA` — nota baixa é chamado, não vitrine. E o importado que a equipe desligou no painel **não reacende** na rodada seguinte.

Sem as chaves nada quebra: o comando e o botão explicam quais três variáveis faltam em vez de chamar a API no escuro. Quando o token expira, o painel devolve o aviso vermelho com a resposta do Google e a linha `depoimentos_google_recusados` fica em *Auditoria*. Para manter o carrossel fresco sem ninguém precisando lembrar, o `schedule` pode chamar o comando uma vez por semana:

```
# cPanel → Cron Jobs, uma vez por semana
cd /home/USUARIO/public_html && php artisan depoimentos:google --limite=50 >>
storage/logs/depoimentos-google.log 2>&1
```

Depoimento escrito pelo cliente

No formulário do painel, o campo **Ficha do cliente** lista apenas fichas com conta no portal (`email_portal` preenchido): é ele que faz a linha dizer quem escreveu, do mesmo jeito que o portal faz quando o cliente envia de dentro da própria conta. O portal abre a tela *Deixar um depoimento* para todo cliente autenticado, mas só grava quando a ficha está `status = ativo`; enquanto houver um texto do mesmo cliente aguardando aprovação, a tela avisa em vez de acumular duplicados. O envio nasce `fonte = cliente`, `aprovado = false` e com o `customer_id` assinando — e a equipe recebe o aviso no `contato_email` já apontando para a fila de pendentes.

Migração nova

`2026_10_24_000003_da_origem_ao_depoimento` acrescenta `fonte` (16, default `site`), `externo_id` (191, único), `customer_id` (FK para `customers`, zera se a ficha for removida) e `avatar` (500). É aditiva, tem `down()`, e os depoimentos que já existiam recebem `fonte = 'site'` pelo próprio default da coluna.

---

12. Retorno e confirmação: uma camada só

Toda resposta do sistema ao usuário — o que deu certo, o que impediu a ação e a pergunta antes de um ato destrutivo — sai por um único lugar: `resources/views/partials/avisos.blade.php`, com a folha `public/css/golden-avisos.css`. Nada de caixa fixa no alto da página e nada de `alert()` ou `confirm()` do navegador: o primeiro quebra o layout em três linhas de português, o segundo é cinza do sistema operacional.

- **Feedback** (salvou, falhou, erros de validação) vira notificação *toastr* no

canto superior direito. O partial lê `success`, `status`, `info`, `warning`, `error` e o `$errors` da view, então qualquer controller que faça `->with('success', …)` ou `->withErrors(…)` já aparece na tela sem código novo.

- **Confirmação** de exclusão, cancelamento e atos parecidos usa **SweetAlert2**.

Basta escrever `data-confirmar="Excluir este usuário?"` no `<form>`; o componente `components/admin/delete.blade.php` já nasce com o atributo, e o interceptor do partial impede o envio até a resposta. Cancelar não navega.

- **Cinco superfícies, uma folha**: painel, portal do cliente, as telas de acesso

(`/login`, `/portal/entrar`, `/portal/cadastro`) e o site público incluem o mesmo partial. No site ele entra com `['confirmacoes' => false]`, porque nenhuma tela pública tem ato destrutivo — assim o SweetAlert2 não é nem baixado.

- **Tema**: a camada de aviso não cria cor própria. Ela consome os tokens de

`golden-theme.css` (`--login-surface`, `--t-*-fg`, e os dois novos `--aviso-sombra` e `--aviso-veu`), então o *toastr* escuro/claro acompanha o botão de tema. Por isso o site público passou a carregar `golden-theme.css` antes de `golden-site.css`.

- **O ícone do toastr**: a folha do CDN pinta um *sprite* de fundo em cada

notificação, e a regra dele usa `!important` com a mesma especificidade da nossa. A trava (`#toast-container>.toast{background-image:none!important}`) mora num `<style>` do próprio partial, logo depois do `<link>` do CDN, porque num empate de especificidade vence quem chega por último ao documento — assim um navegador com a stylesheet antiga em cache não reabre o mosaico de ícones. O desenho do tom (borda + glifo Font Awesome) continua em `golden-avisos.css`, que não declara `background-image` nenhum.

- **Português**: o Laravel 13 não traz texto de validação no repositório, então a

chave crua (`validation.required`) aparecia na tela em vez da mensagem. As traduções estão em `lang/pt_BR/` (`validation`, `auth`, `pagination`, `passwords`, com o mapa `attributes` que dá nome brasileiro a cada campo) e o instalador escreve `APP_LOCALE=pt_BR`. Não há nada extra a fazer na instalação; se você trocar o idioma, troque também `APP_LOCALE` e crie a pasta correspondente.

- **Travado por teste**: `tests/Feature/RetornoEConfirmacaoTest.php` exige que

toda página carregue o partial, proíbe `alert(` / `confirm(` em Blade e no JS público, proíbe `{{ session(… }}` / `{{ $errors->… }}` pintando caixa, e confere que a folha de avisos só usa token existente nas duas paletas. `tests/Feature/TraducoesPtBrTest.php` varre as regras de validação usadas em `app/` e reclama se alguma não tem texto em português.

Instalou e ainda apareceu `validation.required`? É cache de view antigo: `php artisan optimize:clear`.

---

13. Backup e restauração

```
# banco
mysqldump --single-transaction -u USUARIO -p NOME_DO_BANCO | gzip > banco-$(date
+%F).sql.gz
# arquivos enviados
tar czf storage-$(date +%F).tar.gz storage/app/public
```

Restaurar:

```
gunzip < banco-2026-10-23.sql.gz | mysql -u USUARIO -p NOME_DO_BANCO
tar xzf storage-2026-10-23.tar.gz
php artisan storage:link
```

Guarde o `.env` fora do repositório (ele tem a `APP_KEY` e as credenciais): sem ele, os dados criptografados e as sessões não voltam.

---

14. Problemas comuns

Sintoma	Causa	Solução
Erro 500 com `Your Composer dependencies require a PHP version ">= 8.3.0"`	o PHP **da web** é antigo, mesmo com o `php -v` do SSH mostrando 8.3	MultiPHP Manager → 8.3; se o seletor da conta não chegar lá, `--php-web=ea-php83` / a linha `AddType` no `.htaccess` (seção 2)
Site mostra a lista de arquivos e pastas	docroot na raiz sem `.htaccess`, e `Options Indexes` ligado	rode o instalador (ou copie o `.htaccess` da seção 4.7)
`Route [admin.xxx] not defined` ao salvar	cache de rotas velho	`php artisan route:clear && php artisan route:cache`
Página quebra depois de editar um `.php` em `config/`	cache de configuração	`php artisan config:cache`
Imagem enviada não aparece	`public/storage` inexistente	`php artisan storage:link`
Imagem enviada existe no servidor e a URL devolve **403**	o `.htaccess` da raiz bloqueia o caminho `/storage`	tire `storage` da regra de pastas de código-fonte (seção 4.7)
Upload falha com erro em `getimagesize` / `finfo`	`fileinfo` desligada no PHP da web	ative a extensão no seletor de PHP (seção 2)
Upload falha no meio com "passou do limite do servidor"	`post_max_size` menor que o campo	ajuste no painel de PHP (40 MB para vídeo + margem)
`could not find driver`	`pdo_mysql` desligado	ative a extensão no painel de PHP
`500` com `Permission denied` em `storage/`	permissões	`chmod -R 775 storage bootstrap/cache`
`Access denied for user`	usuário sem privilégio no banco	ligar usuário ao banco no painel MySQL
E-mail não sai	`MAIL_MAILER=log` ou credencial errada	conferir `storage/logs/email.log`; depois configurar SMTP
Aparece `validation.required` na tela em vez da mensagem	falta a tradução da framework, ou há cache de view velho	`php artisan optimize:clear`; confira `APP_LOCALE=pt_BR` e a pasta `lang/pt_BR/` (seção 12)

Sintoma	Causa	Solução
`550 ... account or domain may not exist` no `failed_jobs`	o destinatário é um endereço que não existe fora do exemplo (domínio sem MX, como `@exemplo.com.br`)	esperado em conta de demonstração com e-mail fictício; o aviso à equipe saiu, o que não existe é a caixa de exemplo. Troque o `email_portal` da ficha por um endereço real e o ciclo fecha
Falha de login no SMTP com senha certa no `.env`	`MAIL_PASSWORD` sem aspas e com `#`	escreva `MAIL_PASSWORD="..."` — sem aspas o `#` vira comentário
Página do site antiga depois de publicar	`opcache` do servidor	`php artisan optimize:clear`; em cPanel, reinicie o PHP
A senha do painel "não funciona" no servidor	a senha instalada é forte, não é `golden123`	leia em `~/backups-golden/acesso-painel.txt` ou defina outra com `php artisan app:define-senha`
O cliente não consegue entrar no `/portal`	o login do portal é o `email_portal`, não o e-mail da ficha do CRM	conferir o selo *conta ativa* na ficha (Admin → Clientes) e usar aquele e-mail; sem conta, ele cria em `/portal/cadastro`
O link de redefinir a senha morre no caminho	expira em 60 minutos e a assinatura depende da `APP_KEY`	reenviar por `/portal/esqueci-senha`; se a `APP_KEY` foi trocada, os links anteriores deixam de valer (é o esperado)
Contrato anexado não aparece no portal	o status está como `rascunho`	passar para assinado/vigente — rascunho é invisível para o cliente por projeto
O PDF do contrato abre 404 no portal	arquivo fora de `storage/contratos/`, ou o download tentou por URL direta	reanexar pelo dropzone e usar `/portal/contratos/{id}/arquivo`, que é a única porta de saída

Os logs ficam em `storage/logs/laravel.log`; no cPanel também existe `public/error_log`, que o `.htaccess` bloqueia para visitantes, e o erro real continua visível para você pelo painel ou SSH.

---

15. Segurança

- `.env` com `chmod 600` e fora do git; `APP_DEBUG=false` em produção.
- Senha do painel forte e única; sem `golden123` em servidor público.
- **Onde ela fica**: o instalador escreve login e senha em

`~/backups-golden/acesso-painel.txt` (permissão `600`, fora do docroot e do git). A tela de `/login` só mostra a dica de demonstração com `APP_ENV=local` — em produção a tela não entrega credencial nenhuma, e é assim que tem de ser.

- **Perdeu a senha**: pela SSH, sem precisar do painel,

`php artisan app:define-senha admin@golden.com.br 'nova-senha'` (a senha também pode ser digitada no prompt, sem eco). Use `--reativar` se a conta foi desligada. O comando lista os e-mails cadastrados quando o e-mail está errado e grava o evento na trilha de auditoria.

- Repositório público: nenhuma credencial, chave SSH ou cópia de banco entra nele.
- O `.htaccess` bloqueia código, segredos e logs; a entrega real é sempre pela

pasta `public/`.

- Arquivo enviado vira nome legível + sufixo aleatório, na pasta do próprio

campo; a extensão é a do conteúdo, então um script batizado de `.jpg` é recusado em vez de virar `.php` executável.

- O destino do envio (`campo`) decide a pasta; o navegador nunca escolhe caminho

no disco, e o descarte só apaga arquivo que comece na pasta do próprio campo.

- Backups de banco ficam fora do docroot, com `600`.
- Portal do cliente: sessão em chave própria (`login_cliente`), recurso de outro

cliente devolve **404** e não 403, `password` e `remember_token` estão em `$hidden` da ficha, o documento é só leitura no portal (é a chave do vínculo com pedido e contrato), e o cadastro tem honeypot, throttle por rota e consentimento LGPD obrigatório.

- O PDF do contrato não tem URL pública: sai só por `/portal/contratos/{id}/arquivo`,

depois de conferir o dono e que não é rascunho. O caminho gravado no formulário é revalidado contra a pasta do campo de envio.

---

16. Checklist depois de instalar

- [ ] `https://seudominio.com.br` abre e o menu funciona
- [ ] PHP da web conferido (`_chk.php` temporário, ou o aviso final do instalador)
- [ ] `/loja` lista produtos; página de produto mostra imagem e vídeo
- [ ] Checkout registra pedido e ele aparece no painel
- [ ] `/admin` abre com o login criado na instalação
- [ ] Campo de envio: arrastar imagem, vídeo e documento mostra preview e progresso
- [ ] Arquivo enviado pelo painel abre pela URL de `/storage/...` (200, não 403)
- [ ] Teste de barreira: `.env`, `app/` e `storage/logs/laravel.log` respondem 403
- [ ] Configurações do painel com WhatsApp, telefone, e-mail e endereço corretos
- [ ] *Conteúdo → Imagens do site* lista as 19 fotos, e *Conteúdo → Textos das páginas* lista

as sete páginas — trocar uma foto e salvar muda a página pública na hora

- [ ] SMTP preenchido (ou `log`, assumido de propósito)
- [ ] Retorno em português: errar a senha em `/login` deve abrir uma notificação

toastr no canto, nunca `validation.required` e nunca caixa dentro da página

- [ ] Confirmação: excluir um usuário no painel abre o diálogo SweetAlert2, e

Voltar não exclui nem recarrega a lista

- [ ] Cron do agendador: `php artisan schedule:run` a cada minuto (seção 9 — poda do

registro de tráfego **e** fila de e-mails do portal)

- [ ] Fila: abrir um chamado no portal e confirmar que a tabela `jobs` volta a zero no minuto

seguinte — com `MAIL_MAILER=log`, o texto aparece em `storage/logs/email.log`

- [ ] Fila com erro: se `failed_jobs` tiver linha, ler o motivo nela, corrigir e rodar

`php artisan queue:retry all`

- [ ] Tráfego: abrir `/admin/atribuicao` depois de algumas visitas reais e conferir se as

origens aparecem

- [ ] Portal: criar conta em `/portal/cadastro` sobre uma ficha que já existe no CRM e conferir que ela herda os pedidos de convidado
- [ ] Portal: entrar com o `email_portal` (não o e-mail da ficha), abrir um chamado e respondê-lo do painel — a conversa tem de ser a mesma nas duas telas
- [ ] Portal: anexar o PDF de um contrato assinado e baixá-lo pelo portal; com status `rascunho` ele não pode aparecer na lista
- [ ] Backup automático de banco e `storage/app/public`
